



Safe Exam Browser DLL Injection (SEB INJ) Documentation

By Fictiouss

Dec, 17, 2025

Disclaimer!!!

This document is written for educational and informational purposes ONLY.

All content is provided to help readers understand security related things in a learning context.

This material DOES NOT and WILL NOT promote or support any form of unauthorized modification, bypassing of security systems, cheating or misuse of software in any way.

Any actions taken based on the info in this document are done ENTIRELY at the reader's own risk. The author assumes no responsibility for any misuse and violations of terms of service or consequences resulting from the app of this info.

Readers are expected to use this content ethically, legally and responsibly in any way.

CONTENTS

1 Introduction

1.1	Scope of Reverse Engineering and Analysis.	1
1.2	Target Audience.	2
1.3	Goals.	3
1.4	Requirements	3
1.4.1	Skills.	3
1.4.2	Tools.	3

2 Static Analysis

2.1	Virtual Machine.	5
2.1.1	VM Detection Policy.	5
2.1.2	VM Display Error.	9
2.2	Kiosk Mode.	12
2.3	3 rd Party App Control.	15
2.4	Clipboard Policy.	20

3 Exploit Development

3.1	The Injector.	25
3.2	The Native DLL.	27
3.3	The Patcher.	27

left empty intentionally

Introduction

This documentation describes the basic things you have to about reverse engineering a .NET application by doing a 'real-time' testing with a popular application called 'safe exam browser'. And we'll be exploiting it with an automation script. The scripts we'll be using are developed as a security proof of concept to show how static runtime patching works and can alter others memory to do what we want and not what it supposed to do.

This documentation is designed to be a study material by defining things like hooking mechanisms, static analysis and runtime patching. Similarly I tried to make it clear when defining a complicated part by going deep to make it as beginner friendly as possible and to 'indirectly' answer things like how the researcher need to implement these patches or how the developer needs to defend against these kinda exploits.

1.1 Scope of Reverse Engineering and Analysis

The scope focuses on safe exam browser or SEB which is a .NET app so most things you see are going to be in human readable languages unlike most native app where you usually get assembly and only a 'guessed' C code. Our analysis is divided into three parts which are going to be static, dynamic and finally intermediate language redirection (just to make it three).

The scope of the static analysis is crazy enough that it's so easy that anyone who can understand the syntax and make sense of it will get it in like no time. The reason I'm saying this is that safe exam browser is open source so all the source code can be get in no time. The other thing is that even if it wasn't even an open source it's a .NET app so it can be decompiled back to an almost 1 to 1 replica of the original code (assuming it's not obfuscated) thanks to tools like dnspy and it can easily be modified and patched once the target logic is found.

The next scope is dynamic analysis. This includes the 'searching' of the asynchronous procedure call queue and the common language runtime (CLR) hosting environment and if you don't know anything about them don't worry just for now think of them as different ways to run or inject code inside a running process. To make it short, the main purpose of this is to patch things at runtime memory.

Finally, this one is kinda "not useful" but I have got to have three, right? anyways, the analysis focuses on the managed IL layer for redirection of methods called like kiosk mode, virtual machine detection and application whitelisting (which you will see in the next topic).

This documentation **does not** cover the complete reverse engineering of the windows application or let alone the .NET framework but rather focuses on the specific "hook points" where our script intercepts and modifies behavior to enforce things we want to have.

1.2 Target Audience

The primary audience for this documentation are personally going to be security researchers and software devs. This documentation is also structured with newcomers to binary exploitation and reverse engineering in mind. Furthermore, this documentation can be read by anyone who has a passion and dedication in this field.

In addition to whatever thing I said, it provides clear explanations in an 'average' way so that readers at different experience levels can read it.

1.3 Goals

The main goal of this documentation isn't to show you how to exploit or even how to patch as a dev. and yeah, I know that sounds weird but that isn't my main focus. Rather, it's to help you understand things in a slightly fun way and actually use your brain for once. I can't lie, goals like learning how to patch are included but helping you read my beautiful documentation and understand things on your own is my main goal. #fuckAI but we can't really do anything about it so.... thanks for giving it time!

1.4 Requirements

For the requirements, I divided them into two types. One is skills/ experience while the other is tools like the kinda things you need for reverse engineering, dynamic analysis and exploit development. Keep in mind that we'll only be covering a **very small** part of this since we're going to mainly focus on .NET windows applications, specifically safe exam Browser. cause of that, you will need to understand a lot more than what is written in this documentation.

1.4.1 Skills

This documentation is for those who have some basic understanding of reverse engineering (which in our case, isn't very necessary) and exploit development and who also know at least the basic syntax of C# and Cpp. You should also have the dedication to read through the specifications cause we'll be doing that **A LOT** when writing the exploit.

There isn't much to expect since this patch is one of the easier ones. However, in in most situations, you would usually need at least an intermediate level of knowledge in C#, Cpp, assembly, C etc depending on the type of app you're targeting.

1.4.1 Tools

I can't say much about the tools because it mostly depends on your preference but I will be using dnspy for reverse engineering the safe exam browser DLL file and visual studio 2026 for exploit development. Once again, keep in mind that in real situations you will need many more tools like ghidra/ida pro, x64dbg, process hacker and detect it easy.

Static Analysis

I don't have the time and dedication to start from the very beginner stuff. If you want to look at it, you can see:

https://en.wikipedia.org/wiki/Static_program_analysis

<https://medium.com/@tr15t4n/intro-to-net-reverse-engineering-c54823b22d6f>

<https://infosecwriteups.com/fundamentals-of-net-decompilation-with-dnspy-e7456707b1d2>

Note: These resources were found in just 1 min and you have to go way deeper on your own but for now you don't need to know much about reverse engineering for two main reasons:

- It's open source and you don't need to know much for this specific 'exploit' we're developing
- I will be explaining a lot of things while considering a newcomer to this field

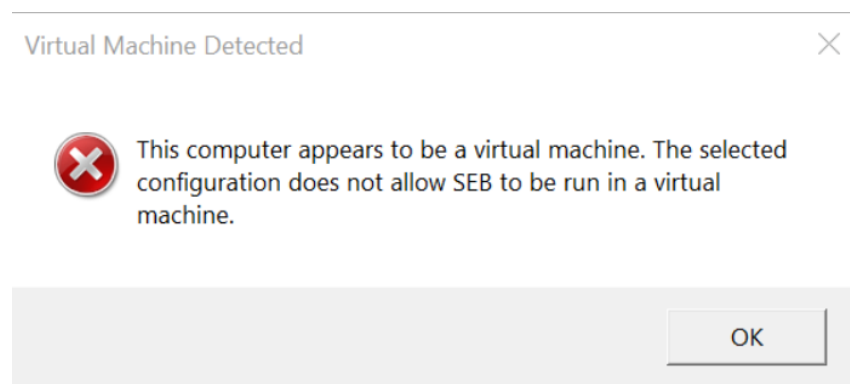
Also, I'll be using a virtual machine (for obvious reasons) but you can test all the exploit/modded versions on any device you want, as long as you know what you're doing and what the code will do.

2.1 Virtual Machine

If you try to open SEB on a virtual machine, it won't open and that's cause of two things that don't allow it to run on a vm. we will be looking at how we can patch both so we can make our modded version of SEB run on a virtual machine and go from there to the other patches.

2.1.1 VM Detection Policy

First, let's try to open SEB on a virtual machine and see what things we can take for our analysis. If you open it, you will see a pop up saying that it detected the virtual machine and there's *no way* of doing it.



Now we have two choices:

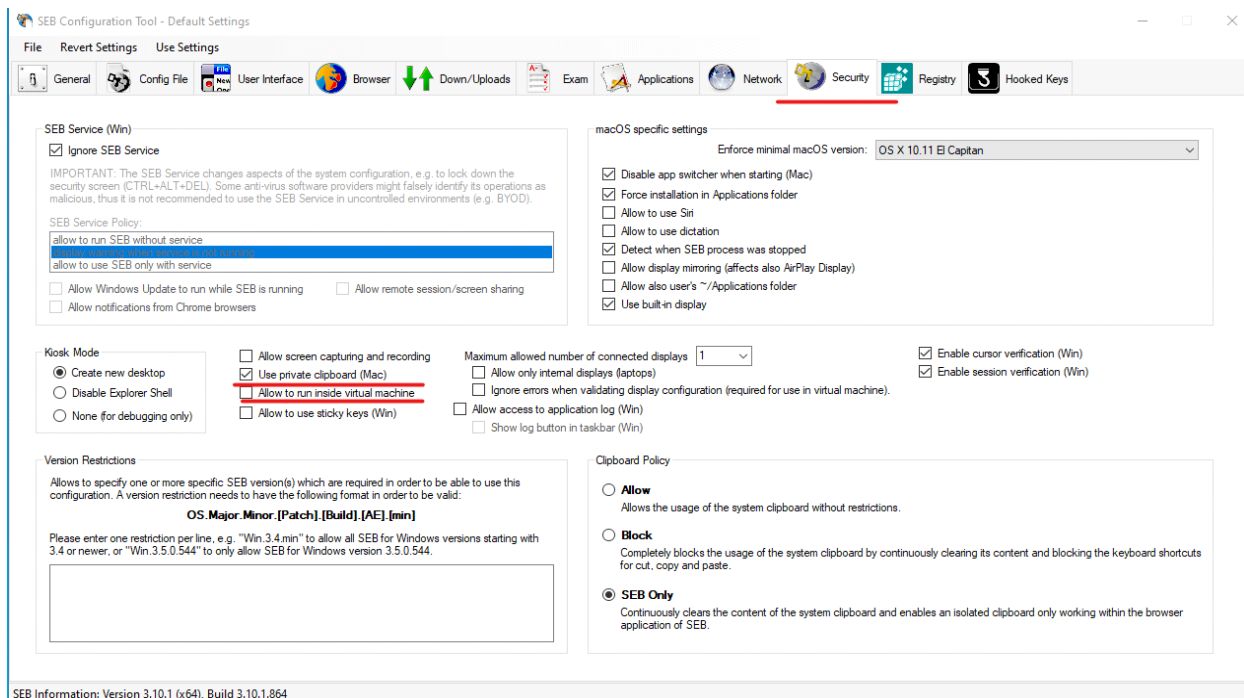
- we can analyze the source code immediately, which is dumb
- we can see if we can 'bypass' it with its own intentional config, which is also dumb but at least a little less dumb, I guess

If you know something about SEB, it has another executable for changing the config for how SEB should run and if we can find anything that has something to do with running on a virtual machine then we can go from there... but for now let's focus on finding it first.

In a 64 bit win, you can find it at:

`C:\Program Files\SafeExamBrowser\Configuration\SEBConfigTool` and if you do a little bit of searching you should see a checkbox with 'allow to run in a virtual machine' beside it in a

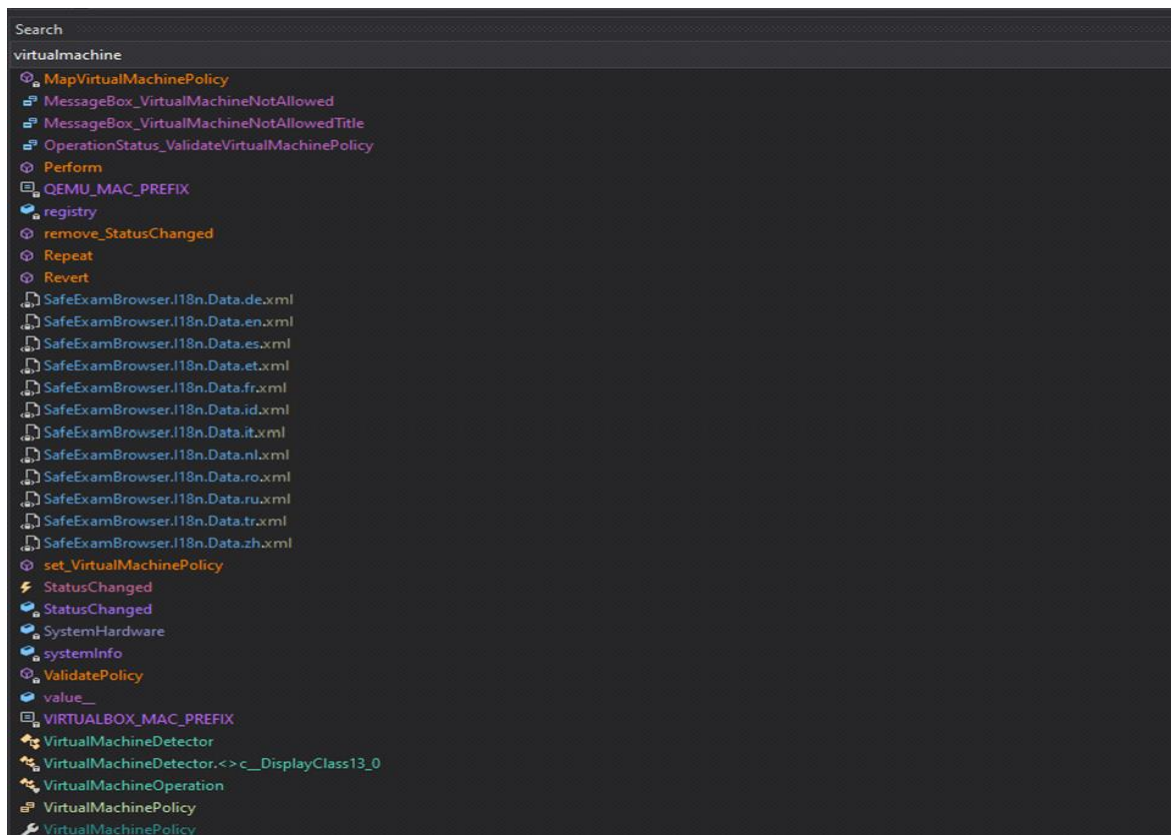
security tab.



So our goal is to always make it 'checked' and we could patch *SEBConfigTool.exe* to always write to the config as if it's checked **BUT** that's a messed up thing and you will know why in the next part. So our only choice would be to patch the SEB's dll to ignore what value is set in the config and always return 'checked'/allowed and to do that we have to use a tool called dnspy (which you can download from <https://github.com/dnSpy/dnSpy/releases>).

I mean, we can just use the original source code (from <https://github.com/SafeExamBrowser/seb-win-refactoring>) but where's the fun in reverse engineering if we can't use the tools, right? but at the end of the day, it's all about learning new things and tools.... do what you think is best.

Now, to find the right logic, we have to decompile all the dlls and executables in *C:\Program Files\SafeExamBrowser\Application* and search for strings that we think will lead us to the target logic (vm detection).



There are two ways we can patch this:

- By either doing it all manually, which is going to take us a while
- By returning the same value

So if we check each one by one, you will find the two things that can get returned based on the config that was set

So now we know that VirtualMachinePolicy.Allow is set as if the checkbox is selected, while VirtualMachinePolicy.Deny is set when it's not allowed for the vm. so if we can return VirtualMachinePolicy.Allow without checking what the config set, we'll be able to manipulate it and get the same result despite what the configuration was.

Now the questions is how, right?

If we can 'analyze' VirtualMachinePolicy enumeration by using dnspy 'analyze' tool we can find who used this and hopefully find the logic that returns one of those

```

318         bool disableSessionChangeLockScreen = (bool)value;
319         settings.Security.DisableSessionChangeLockScreen = disableSessionChangeLockScreen;
320     }
321 }
322
323 // Token: 0x06000157 RID: 343 RVA: 0x000AD10 File Offset: 0x0008F10
324 private void MapVirtualMachinePolicy(AppSettings settings, object value)
325 {
326     if (value is bool)
327     {
328         bool flag = (bool)value;
329         settings.Security.VirtualMachinePolicy = (flag ? VirtualMachinePolicy.Allow : VirtualMachinePolicy.Deny);
330     }
331 }
332
333 // Token: 0x06000158 RID: 344 RVA: 0x000AD40 File Offset: 0x0008F40
334 private void MapReconfigurationUrl(AppSettings settings, object value)
335 {
336 }
337 }

```

Analyzer

- VirtualMachinePolicy @02000018
 - Exposed By
 - Extension Methods
 - Used By
 - SafeExamBrowser.Configuration.ConfigurationData.DataMapping.SecurityDataMapper.MapVirtualMachinePolicy(AppSettings, object) : void @06000157
 - Used By
 - Uses
 - SafeExamBrowser.Configuration.ConfigurationData.DataValues.LoadDefaultSettings() : AppSettings @060000C6
 - SafeExamBrowser.Runtime.Operations.Session.ConfigurationOperation.HandleBrowserResource(Un) : OperationResult @0600007C
 - SafeExamBrowser.Runtime.Operations.Session.VirtualMachineOperation.ValidatePolicy() : OperationResult @06000107
 - SafeExamBrowser.Settings.Security.SecuritySettings.<VirtualMachinePolicy>k__BackingField : VirtualMachinePolicy @04000078
 - SafeExamBrowser.Settings.Security.SecuritySettings.VirtualMachinePolicy : VirtualMachinePolicy @1700005D

and we found MapVirtualMachinePolicy method which is the main core for returning the final result so instead of letting it check what the config is set we ‘force’ it to always return VirtualMachinePolicy.Allow so it can run on vm despite the config setup.

(I’m assuming you know how to edit code’s if not that’s on you)

now, we have to edit settings.Security.VirtualMachinePolicy to be ‘predectable’ by setting it to the allowed and making ignore what the configuration command it to do

```

1  using System;
2  using System.Collections.Generic;
3  using SafeExamBrowser.Settings;
4  using SafeExamBrowser.Settings.Security;
5
6  namespace SafeExamBrowser.Configuration.ConfigurationData.DataMapping
7  {
8      // Token: 0x02000025 RID: 37
9      internal partial class SecurityDataMapper : BaseDataMapper
10     {
11         // Token: 0x06000131 RID: 305 RVA: 0x000A8D0 File Offset: 0x0008AD0
12         private void MapVirtualMachinePolicy(AppSettings settings, object value)
13         {
14             if (value is bool)
15             {
16                 bool allow = (bool)value;
17                 // settings.Security.VirtualMachinePolicy = (allow ? VirtualMachinePolicy.Allow : VirtualMachinePolicy.Deny);
18                 settings.Security.VirtualMachinePolicy = 0;
19             }
20         }
21     }
22 }
23

```

NOTE: 0 is for VirtualMachinePolicy.Allow while 1 is for deny

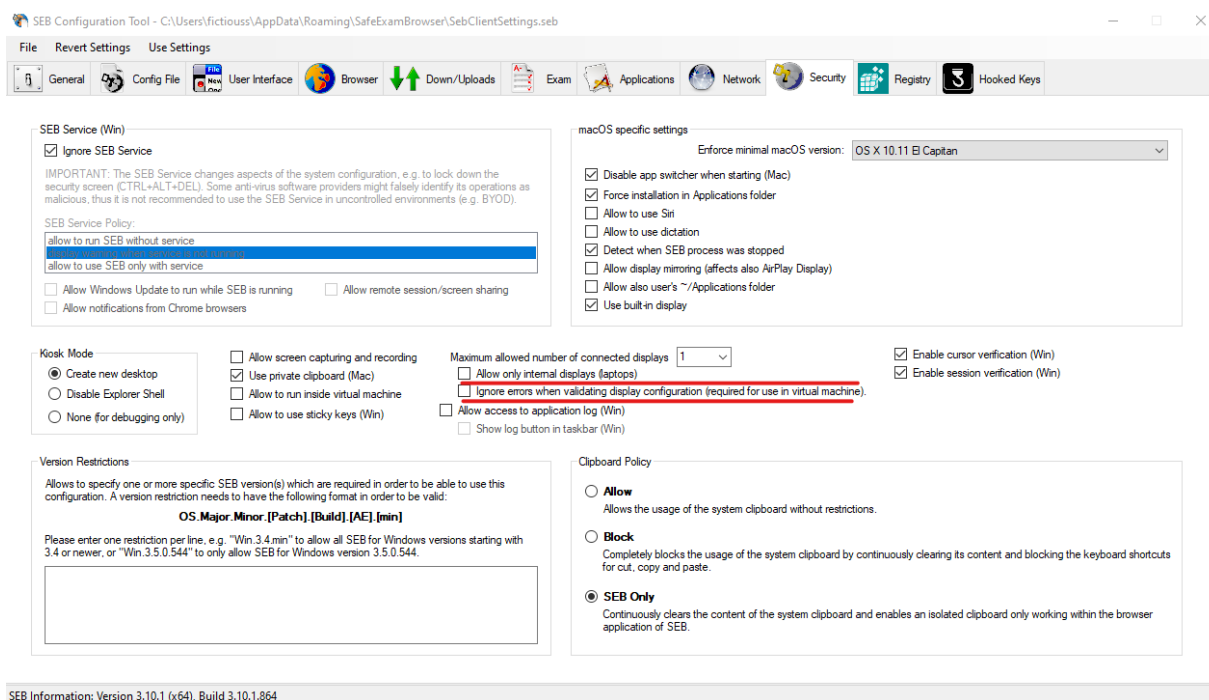
once we compiled everything and exported the new patched dll, we then replace the

original one with and test if everything works, fails or we need some other patches too also I need to remind you that always and always unless you're 100% make a backup of the original dll executable you want to patch so in that cause you don't have to reinstall the whole thing if anything messed up or you ended up in a rabbit hole and wants to start over.

and if we run it you will see the pop up dialog doesn't appear but instead a new dialog pops up which this time saying "Active display configuration isn't permitted..." which tells that we successfully bypassed the vm detection but something else is still stopping us from running seb in a virtual machine and that's where we go to the next section about VM Display Error.

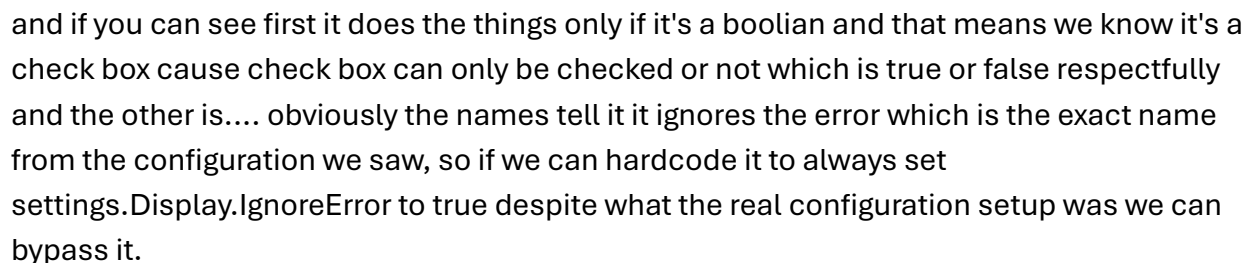
2.1.2 VM Display Error

As earlier we have to go and look at the configuration setup and see if we can find this specific and use it's own functionality to bypass the virtual machine detection. And if you look at the 'security' tab (the same tab we found the vm detection setting on)



If you look at the obvious checkbox you will see "Ignore errors when validating... (required for use in virtual machine)" so if we can do the same patch to this one like we did with the vm detection we should be (will since I know what's gonna happen lol) able to run SEB in a virtual machine and do the next patches from there. yeah also I don't think you dumb to

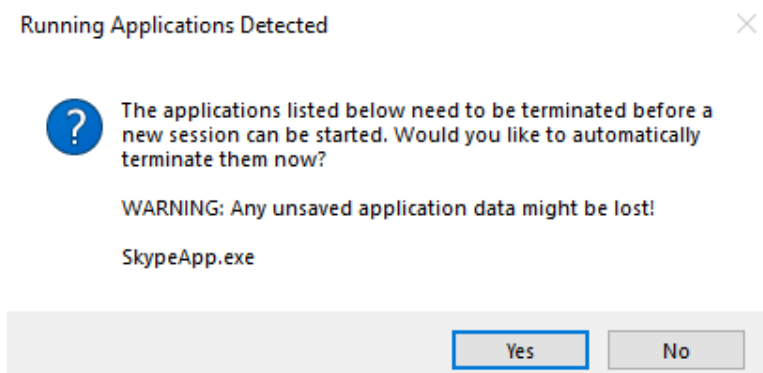
anyway, just like earlier we can 'guess' what would it be since it's not obfuscated so we can search for things like 'display', 'error', 'ignore errors', 'ignore display', 'active display error', or even 'Virtual machine display error' if you want to have a small range but I advise you to go as broad as possible unless you're sure cause it's good to have a lot of choice than nothing. in this I will be searching for 'display error' (don't ask why, cause that was the first thing that came to my mind) and if we do search on it we find not that much but at the same time not that small amount of results and what I did here is analyze each and every single search until I find something that makes sense to control the vm active display error handling.



NOTE: you can lower up the range by only searching for methods which is what we want in most non-obfuscated application

```
Edit Code - MapIgnoreError(AppSettings, object) : void @0600011A
1 using System;
2 using SafeExamBrowser.Settings;
3
4 namespace SafeExamBrowser.Configuration.ConfigurationData.DataMapping
5 {
6     // Token: 0x02000054 RID: 84
7     internal partial class DisplayDataMapper : BaseDataMapper
8     {
9         // Token: 0x0600011A RID: 282 RVA: 0x00009BE8 File Offset: 0x00007DE8
10        private void MapIgnoreError(AppSettings settings, object value)
11        {
12            if (value is bool)
13            {
14                bool ignoreError = (bool)value;
15                //settings.Display.IgnoreError = ignoreError;
16                settings.Display.IgnoreError = true;
17            }
18        }
19    }
20 }
21
```

After that patch everything should be go to go and be able to open on virtual machine despite of what the configuration was... but if you notice something while you opened seb and some other application were running and in my case it was skype seb won't open unless that application is close and will pop up something like the image you see



this won't matter for now but I just want you to remember that cause we are going to come back to it later in the next section.

Anyway, now that we're done with vm detection and display errors we can move on to Kiosk Mode patching which is the main part of this thing in the first place.

2.2 Kiosk Mode

Kiosk Mode is a mid to high security system state that restricts the windows operating environment to a single application window by preventing access to unauthorized files or/and third-party software or system tools like ALT combo. So in a shortest way it converts multi-purpose computer into a single-purpose 'application'.

If you have no idea on what I said and wants to go deeper, see

https://en.wikipedia.org/wiki/Kiosk_software and others, it's up to you.

Anyway, now that you have some understanding of what kiosk mode can do, let's go back to safe exam browser and talk about how we can 'bypass' it with it's own configuration. we're going to use the same strat again!... yeah it's that easy, we just go to configuration and find anything related to setting kiosk mode and hardcode the value (return an altered value instead of the original one).

The screenshot shows the SEB Configuration Tool window with the title bar "SEB Configuration Tool - C:\Users\fictionous\AppData\Roaming\SafeExamBrowser\SebClientSettings.seb". The interface includes a menu bar (File, Revert Settings, Use Settings) and a toolbar with icons for General, Config File, User Interface, Browser, Down/Uploads, Exam, Applications, Network, Security, Registry, and Hooked Keys. The main content area is divided into several sections:

- SEB Service (Win):** Includes a checkbox for "Ignore SEB Service" (checked). Below it is an important note about system configuration changes. A "SEB Service Policy" dropdown menu is set to "allow to use SEB only with service". There are also checkboxes for "Allow Windows Update to run while SEB is running", "Allow remote session/screen sharing", and "Allow notifications from Chrome browsers".
- macOS specific settings:** Features a dropdown for "Enforce minimal macOS version" set to "OS X 10.11 El Capitan". It includes several checkboxes: "Disable app switcher when starting (Mac)" (checked), "Force installation in Applications folder" (checked), "Allow to use Siri" (unchecked), "Allow to use dictation" (unchecked), "Detect when SEB process was stopped" (checked), "Allow display mirroring (affects also AirPlay Display)" (unchecked), "Allow also user's ~/Applications folder" (unchecked), and "Use built-in display" (checked).
- Kiosk Mode:** A red box highlights this section. It contains three radio buttons: "Create new desktop" (selected), "Disable Explorer Shell" (unchecked), and "None (for debugging only)" (unchecked). To the right are checkboxes for "Allow screen capturing and recording" (unchecked), "Use private clipboard (Mac)" (checked), "Allow to run inside virtual machine" (unchecked), "Allow to use sticky keys (Win)" (unchecked), "Maximum allowed number of connected displays" (set to 1), "Allow only internal displays (laptops)" (unchecked), "Ignore errors when validating display configuration (required for use in virtual machine)" (unchecked), "Allow access to application log (Win)" (unchecked), "Show log button in taskbar (Win)" (unchecked), "Enable cursor verification (Win)" (checked), and "Enable session verification (Win)" (checked).
- Version Restrictions:** A text box for specifying SEB version restrictions. It includes a note about the format "OS.Major.Minor.[Patch].[Build].[AE].[min]" and an example: "Win.3.4.min".
- Clipboard Policy:** Includes three radio buttons: "Allow" (unchecked), "Block" (unchecked), and "SEB Only" (checked). The "SEB Only" option is described as "Continuously clears the content of the system clipboard and enables an isolated clipboard only working within the browser application of SEB."

At the bottom, the status bar reads: "SEB Information: Version 3.10.1 (x64), Build 3.10.1.864".

so there are 3 check boxes

1. Create new desktop: is the default the max security kiosk mode
2. Disable explorer shell: is the same as 'create new desktop' but it doesn't create a new desktop instead it runs seb in the existing current desktop workspace and only closes all the application and kills the explorer but things like hotkey works.
3. None: will make seb just like other normal windows application.

Of all those three our target is going to be 'Disable explorer shell' and why you asked? I'm gonna tell you why not others

create new desktop: we need to modify A LOT of things one by one... which includes allowing each hotkey, making it run on the same windows workspace (not really necessary), clipboard allowance between seb and third party application etc all to reach 'disable explorer shell' mode which is what I want to think is the dumb way and waste of time... but if you really want to learn more and not skip anything I really recomend you following this step cause I'll tell you you're not going to regret doing all that, but for the sake of this documentation I'll be doing it the easiest way possible.

The 'NONE' is perfect... until you see what the gui looks like, I mean unless you want it for like a personal thing or small stake exam to take in home, this won't work and it would be easier to just call your mentor and tell yourself it's cracked lol.

WARNING: ONCE AGAIN I DO NOT PROMOT ANY TYPE OF CHEATING THAT WAS JUST FOR EXAMPLE AND DON'T BE JERK TRYING TO CHEAT.

now since we know our goal, let's go back to reverse engineering.

as always we search for string and in my case I searched for 'kiosk mode' (just a random choice) and this time only searched for a METHOD instead of all things (classes, fields, property...) and I got like 10-15 results and after reviewing all we can see a method 'MapKioskMode' is the main logic for handling kiosk mode. this method is just a series of "if/then" checks by first checking if it's set to 'kill explorer shell' then 'create new desktop' then if neither of them are set in config it will set it to 'none'.

Despite the code line number being high, it's actually too easy to patch this by either setting flag2 to true or by just removing all "if/then" statments and just outputing the final alterd result which what we're going to do now as you can see below

```

Edit Code - MapKioskMode(Dictionary<string, object>, AppSettings): void @06000153
1  using System;
2  using System.Collections.Generic;
3  using SafeExamBrowser.Settings;
4  using SafeExamBrowser.Settings.Security;
5
6  namespace SafeExamBrowser.Configuration.ConfigurationData.DataMapping
7  {
8      // Token: 0x02000059 RID: 89
9      internal partial class SecurityDataMapper : BaseDataMapper
10     {
11         // Token: 0x06000153 RID: 339 RVA: 0x0000AB4C File Offset: 0x00008D4C
12         private void MapKioskMode(Dictionary<string, object> rawData, AppSettings settings)
13         {
14             settings.Security.KioskMode = KioskMode.DisableExplorerShell;
15             //object obj;
16             //bool flag = rawData.TryGetValue("createNewDesktop", out obj);
17             //object obj2;
18             //bool flag2 = rawData.TryGetValue("killExplorerShell", out obj2);
19             //if (flag2)
20             //{
21                 //bool? flag3 = obj2 as bool?;
22                 //bool flag4 = true;
23                 //if (flag3.GetValueOrDefault() == flag4 & flag3 != null)
24                 //{
25                     // settings.Security.KioskMode = KioskMode.DisableExplorerShell;
26                 }
27             }
28             //if (flag)
29             //{
30                 //bool? flag3 = obj as bool?;
31                 //bool flag4 = true;
32                 //if (flag3.GetValueOrDefault() == flag4 & flag3 != null)
33                 //{
34                     // settings.Security.KioskMode = KioskMode.CreateNewDesktop;
35                 }
36             }
37             //if (flag && flag2)
38             //{
39                 //bool? flag3 = obj as bool?;
40                 //bool flag4 = false;
41                 //if (flag3.GetValueOrDefault() == flag4 & flag3 != null)
42                 //{
43                     //flag3 = (obj2 as bool?);
44                     //flag4 = false;
45                     //if (flag3.GetValueOrDefault() == flag4 & flag3 != null)
46                     //{
47                         //settings.Security.KioskMode = KioskMode.None;
48                     }
49                 }
50             }
51         }
52     }
53 }
54

```

Now, how do we know it really is in 'disable explorer shell' mode and not 'create new desktop' mode cause both looks the same to the point of being completely similar so how do we know our patch works...and that's actually easy to like figure out. you can either open an explorer windows and then open SEB and after you exit seb if it you found the explorer windows closed that means seb opened in disable explorer shell mode but if you are a nerd you can read the recent log and search for kiosk and see which mode it intialized on. you can find the log at %LocalAppData%\SafeExamBrowser\Logs and in the recent log if you find something like:

<date> - INFO: Initializing kiosk mode 'DisableExplorerShell'...
that means everything is working in our favor.

Now that we're done with kiosk mode and we know we're running on the same windows work space, we got to go to the next patchs which we'll talk about it in the next section.

2.3 3rd Party App Control

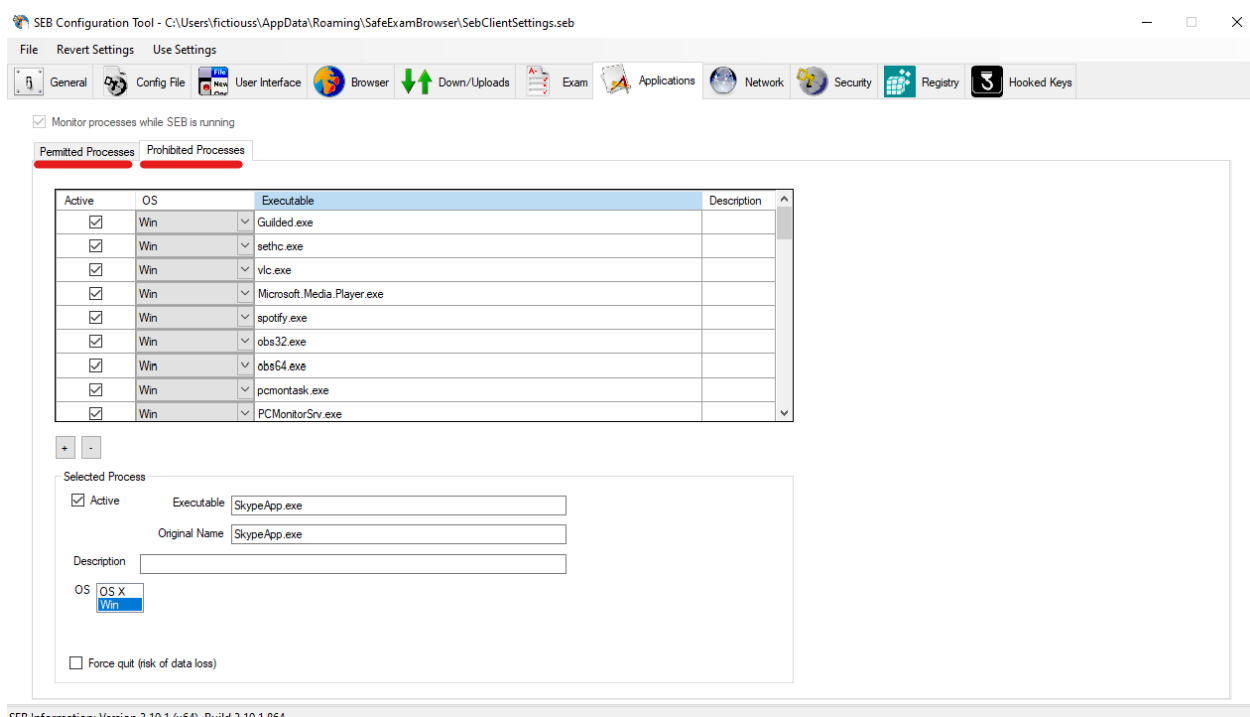
While Kiosk Mode locks the operating system's interface, app control handles the specific processes running in the background... which was I thought in a first place but in SEB it closes those blacklisted 3rd party app on initialization and after that it's fully 'vulnerable' if you had an other 'malicious' process waiting for a user to initialize one of those applications after the validation and initialization is done. SEB have two lists: permitted processes which are the whitelisted one and prohibited processes are in other hand the blacklisted application list and if you're a normal human being you would know what each means but one thing is different here, whitelisted process are not just an application that's not won't closed by SEB but are also the one that will get added to seb application itself and you will understand why I mean by that in the next part for now just know that we'll be using it for things like chrome to run within the browser aka using it's own function for our own purpose.

Blacklisted application can be altered in a configuration by adding or removing a processes but the default one will include any type of communication media (social media), browsers, screen sharing applications, system applications etc. but since it checks and validates only one the initialization stage (startup) we should be good to go, but what we have to consider is that there might be a chance our injector might be on the list, yeah I know it's low chance but we can't take risks so we got to patch it to be sure that there is no prohibited process.

NOTE: this is not a necessary patch for the final result but as long as it's for learning purposes, which hopefully it is, it's good to read and understand this topic.

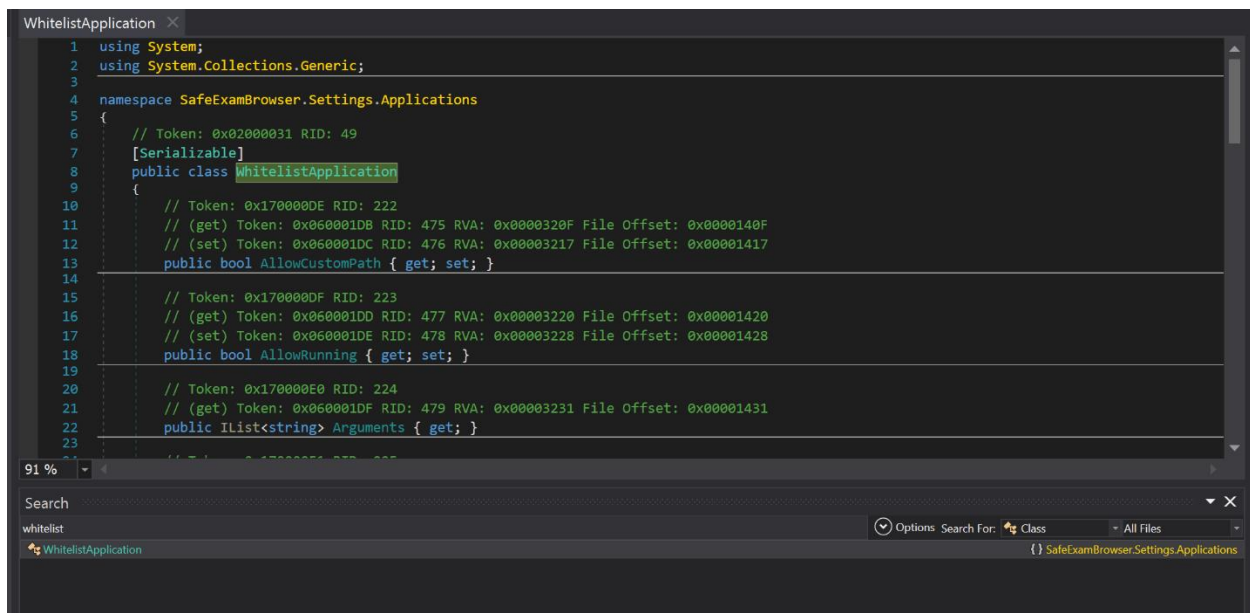
On the other hand, there are whitelisted apps that aren't what you think they are... what I mean by that is that they aren't just allowed but they are also made easy to access within the browser since the taskbar of SEB has the same functionality as a normal Windows taskbar.

Now let's stop with all that yapping and let me go straight to the code and as always we got to look at the configuration and 'guess' what to search for



we got to search for obvious words like "whitelist", "blacklist", or "Allowed" and this pulls up classes like Whitelistapplication inside the SafeExamBrowser.Settings.dll

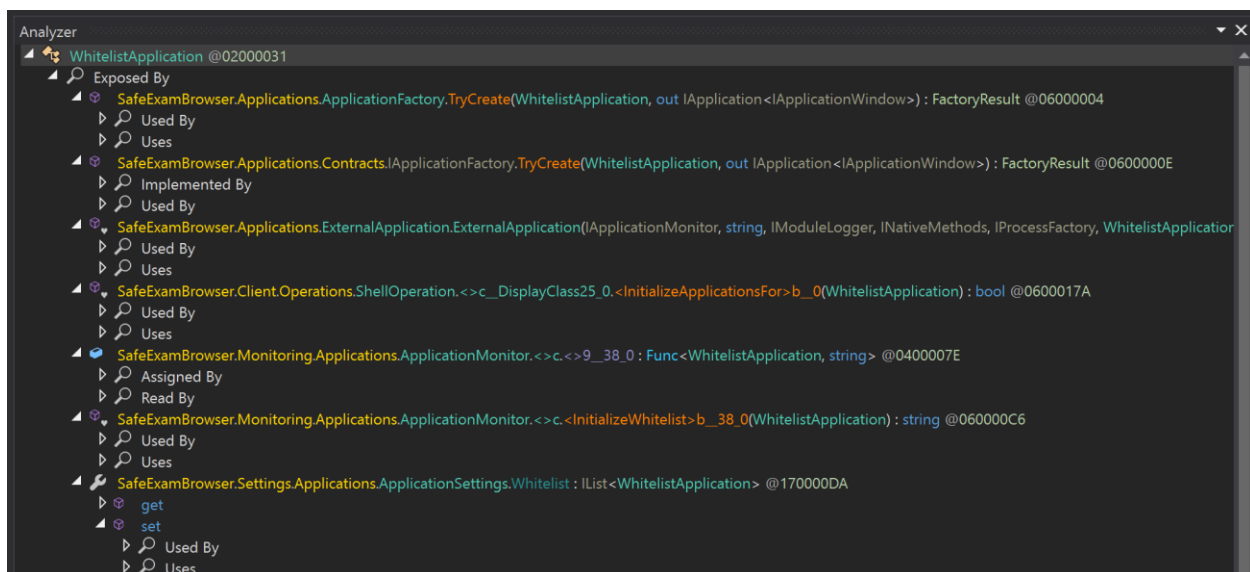
NOTE: if you can't find any method, the best next choice would be to filter for classes just like I did in this one



As you can see in the image, C# devs don't just use a simple text list like ["msedge.exe", "calculator.exe"] to store allowed app... you can store it in simple text but that would be DUMB! so instead, seb needs to store a lot of rules for every single app like what the app's name is (DisplayName), should it automatically open in startup of seb (AutoStart), is the user allowed to run it (AllowRunning) and many more to keep all of this organized, SEB created a class called WhitelistApplication. it is simply a data structure that represents one single allowed app so when seb processes its config file, it creates a new WhitelistApplication object for every app it finds, fills out the details and finally, SHOVE it into the whitelist array.

But this is not what we want, rather we want how we can allow application and clear out the whole blacklisted apps and in order to do that we got to go deeper so the next and most practical thing would be to 'analyze' the class

NOTE: 'analyze' is NOT the same as analyze



Look at the image... do you see anything different among all those classes (advice: this is a 100% **impractical** way of doing things, instead you got to analyze everything (or at least the ones that make sense like method/class name) and to be honest you have to if it's obfuscated and for most apps nowadays it will be)

Anyway, now that we found ApplicationSettings, we can see it holds something and to be specific it has public IList Blacklist { get; set; }.

```

ApplicationSettings X
4 namespace SafeExamBrowser.Settings.Applications
5 {
6     // Token: 0x0200002F RID: 47
7     [Serializable]
8     public class ApplicationSettings
9     {
10         // Token: 0x170000D9 RID: 217
11         // (get) Token: 0x060001CF RID: 463 RVA: 0x00003194 File Offset: 0x00001394
12         // (set) Token: 0x060001D0 RID: 464 RVA: 0x0000319C File Offset: 0x0000139C
13         public IList<BlacklistApplication> Blacklist { get; set; }
14
15         // Token: 0x170000DA RID: 218
16         // (get) Token: 0x060001D1 RID: 465 RVA: 0x000031A5 File Offset: 0x000013A5
17         // (set) Token: 0x060001D2 RID: 466 RVA: 0x000031AD File Offset: 0x000013AD
18         public IList<WhitelistApplication> Whitelist { get; set; }
19
20         // Token: 0x060001D3 RID: 467 RVA: 0x000031B6 File Offset: 0x000013B6
21         public ApplicationSettings()
22         {
23             this.Blacklist = new List<BlacklistApplication>();
24             this.Whitelist = new List<WhitelistApplication>();
25         }
26     }
27 }

```

we want to know who controls this list, since ApplicationSettings is just one category of settings. it is "placed" inside an even bigger box, so to figure that out we got to 'analyze' the class and if we look by which it got "exposed by", we'll see it is being used by another class called AppSettings... you asked how I found it? The answer will be... LOOK AGAIN.

once again, this might make my work a lot easier but in most apps this won't work, you got to analyze each and every class since most will be obfuscated.

```

AppSettings X
11 using SafeExamBrowser.Settings.SystemComponents;
12 using SafeExamBrowser.Settings.UserInterface;
13
14 namespace SafeExamBrowser.Settings
15 {
16     // Token: 0x02000006 RID: 6
17     [Serializable]
18     public class AppSettings
19     {
20         // Token: 0x17000001 RID: 1
21         // (get) Token: 0x06000003 RID: 3 RVA: 0x0000205F File Offset: 0x0000025F
22         // (set) Token: 0x06000004 RID: 4 RVA: 0x00002067 File Offset: 0x00000267
23         public ApplicationSettings Applications { get; set; }
24
25         // Token: 0x17000002 RID: 2
26         // (get) Token: 0x06000005 RID: 5 RVA: 0x00002070 File Offset: 0x00000270
27         // (set) Token: 0x06000006 RID: 6 RVA: 0x00002078 File Offset: 0x00000278
28         public AudioSettings Audio { get; set; }
29
30         // Token: 0x17000003 RID: 3
31         // (get) Token: 0x06000007 RID: 7 RVA: 0x00002081 File Offset: 0x00000281
32         // (set) Token: 0x06000008 RID: 8 RVA: 0x00002089 File Offset: 0x00000289
33     }
34 }

```

91 %

Analyzer

- ApplicationSettings @0200002F
 - Exposed By
 - SafeExamBrowser.Monitoring.Applications.ApplicationMonitor.Initialize(ApplicationSettings) : InitializationResult @0600008D
 - Used By
 - Uses
 - SafeExamBrowser.Monitoring.Contracts.Applications.IApplicationMonitor.Initialize(ApplicationSettings) : InitializationResult @0600004B
 - Implemented By
 - Used By
 - SafeExamBrowser.Settings.AppSettings.Applications : ApplicationSettings @17000001
 - get
 - set

AppSettings is the mega version of ApplicationSettings that holds everything for SEB, things like security settings, browser settings and many more

Now that we have the AppSettings, we need to find what function in seb actually 'builds' it when the program loads up

And if we 'analyze' AppSettings and look at who creates it or uses it, we should get a TON of results but among those the following are the ones that I think you should know, even though only one is going to get patched

SafeExamBrowser.Configuration.ConfigurationData.DataValues (this generates the defaults)

SafeExamBrowser.Configuration.ConfigurationData.DataMapper (this reads the exam config files)

SafeExamBrowser.Configuration.ConfigurationData.DataProcessor (this is the FINAL script that processes everything and the one that we're going to patch)

SafeExamBrowser.Configuration.ConfigurationRepository.TryLoadSettings() (this loads the setting that read from exam config)

Now it's time to patch *DataProcessor.Process(...)* because it's the very last one before the fully built config is set up in SEB, so we know it's the perfect place to patch.

```
Edit Code: Process(IDictionary<string, object>, AppSettings): void @00000093
1  using System;
2  using System.Collections.Generic;
3  using SafeExamBrowser.Settings;
4
5  namespace SafeExamBrowser.Configuration.ConfigurationData
6  {
7      // Token: 0x02000017 RID: 23
8      internal partial class DataProcessor
9      {
10         // Token: 0x06000093 RID: 147 RVA: 0x00005948 File Offset: 0x00003B48
11         internal void Process(IDictionary<string, object> rawData, AppSettings settings)
12         {
13             this.ProcessDefault(settings);
14             this.CalculateConfigurationKey(rawData, settings);
15
16             // First patch: clear all blacklisted apps loaded from the .seb config
17             if (settings.Applications.Blacklist != null)
18             {
19                 settings.Applications.Blacklist.Clear();
20             }
21
22             // The second path: add msedge.exe to the whitelist
23             if (settings.Applications.Whitelist != null)
24             {
25                 settings.Applications.Whitelist.Add(new SafeExamBrowser.Settings.Applications.WhitelistApplication
26                 {
27                     ExecutableName = "msedge.exe",
28                     OriginalName = "msedge.exe", // I think this is required by newer versions of SEB but I'm not sure... it's upto you to test it out
29                     DisplayName = "microsoft edge",
30                     AllowRunning = true
31                 });
32             }
33         }
34     }
35 }
36
37
38
```

NOTICE: you won't be able to see the logo or anything in task bar and side bar cause it comes down to the difference between a security rule (which is what we patched right now) and a ui rule.

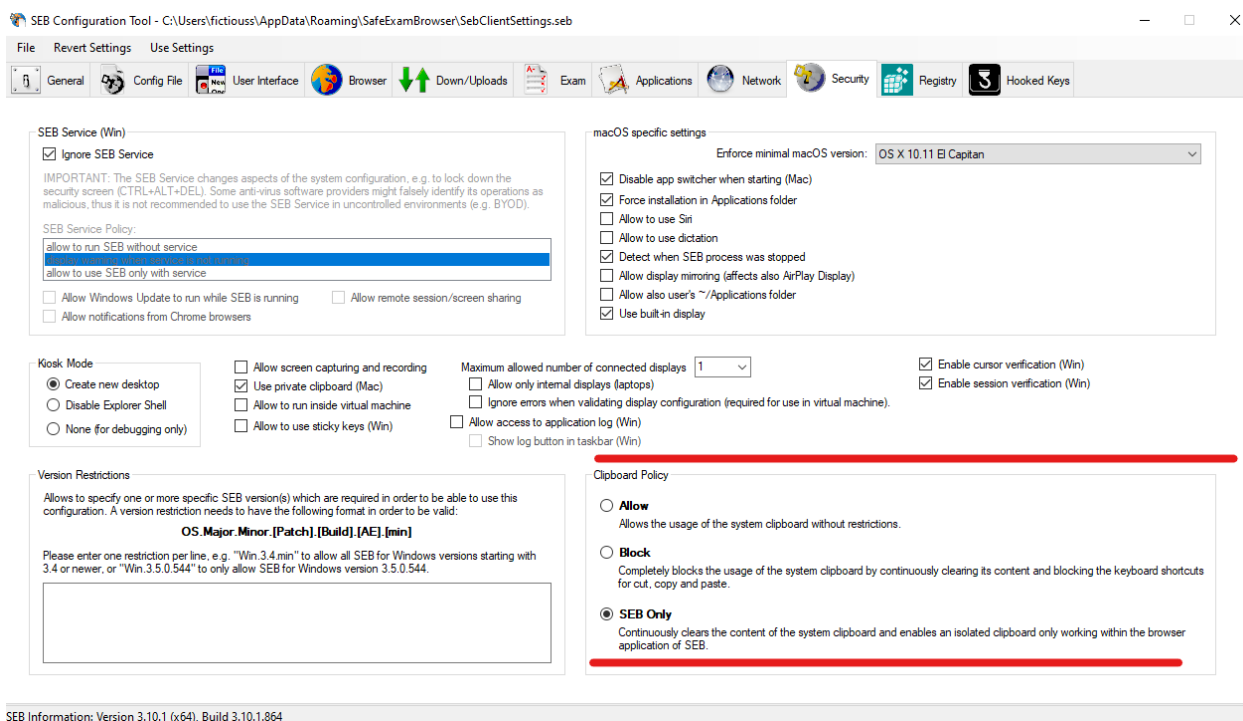
The seb ui (taskbar and sidebar) has its own logic so without any extra patch this property is false because it IS false, the seb shell completely hides the icon from the sidebar since we didn't set it to true in the first place unlike doing it from the configuration which automatically set it to true by default

If you want to fix/patch this and make the logo appear, you need to add ShowInShell = true to your static patch block in `public bool ShowInShell { get; set; }` after obviously... further analyze. But remember you're giving up the look(ui) by doing this.

2.4 Clipboard Policy

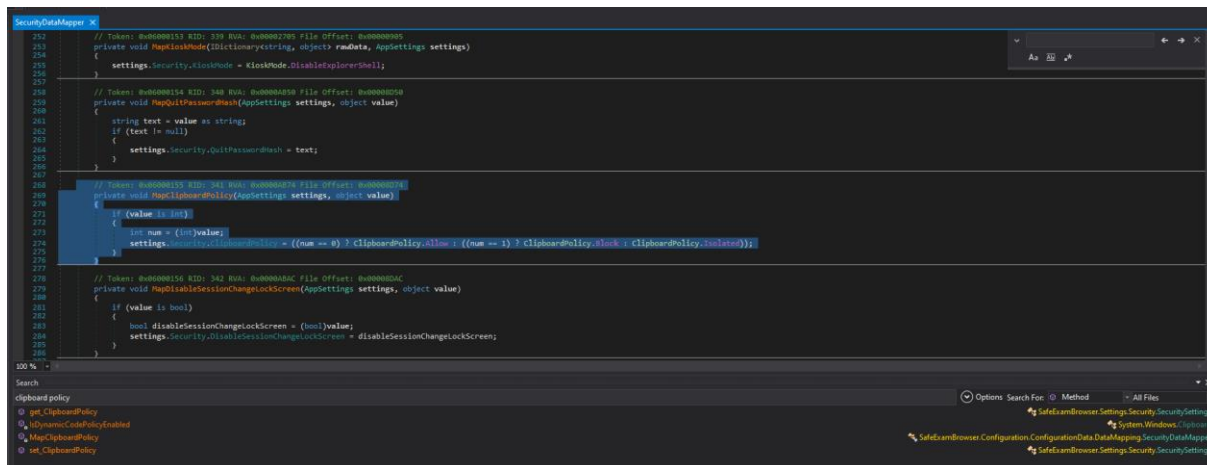
the clipboard policy in SEB is a security feature that obviously and simply stops you from copy-pasting from and within the application. you don't need much info about it cause the title says it all

now as always and always we got to look at the configuration for any hint to search for

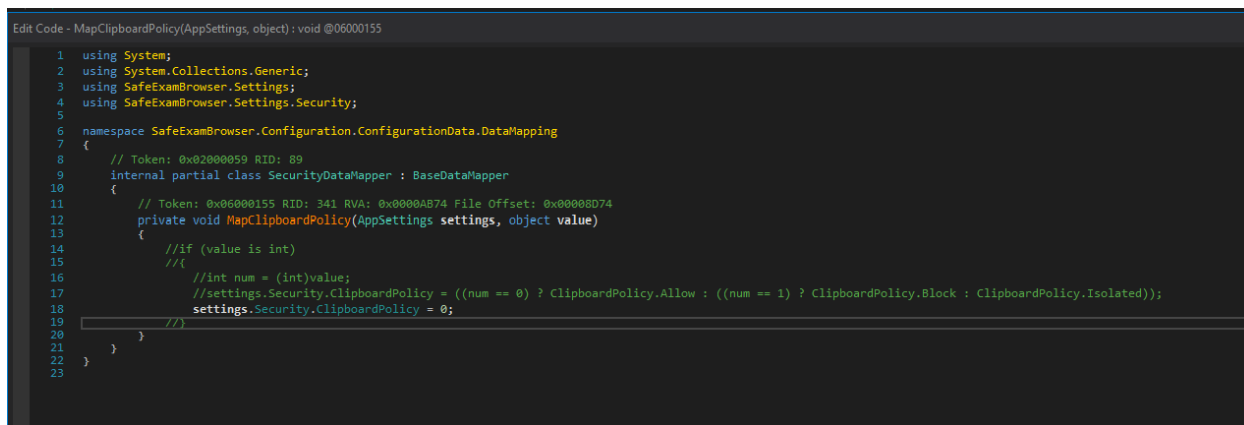


and as you can see it has three things which explain themselves... so we'll be focusing on the 'Allow' one cause that's the one that will give us full clipboard access and the way to do that is going to be the same boring/interesting way by searching for methods with the name

like 'clipboard', 'clipboard policy', 'clip_board' etc but I'm just going to go with the second one cause that's what I think will most likely be named.



at this point you should be able to know which one will have the logic... logical reasoning 101 and after looking at the method MapClipboardPolicy we can be sure that we're on the right one, now let's patch it like before and I'm not going to explain this one either cause I'm pretty sure you know what I'm doing by this point (UNLESS YOU SKIPPED A SECTION, which is on you)



how do you check it up? you can either set the kiosk mode to none and then try copy-pasting from the browser to other third party apps or the easiest nerdy way is just to check the recent logs in %LocalAppData%\SafeExamBrowser\Logs, which I'm going to leave it to you.

Now, that we have and know everything about our goals, we can go to the next level because if you think about it, there are a TON of drawbacks when it comes to static binary patching. The main are:

1. updates
2. browser exam key

if you're thinking what this girl is talking about, don't worry we're going to look at it in the next chapter ;)

Exploit Development

I'm not going to explain what exploit development is and how it works cause there are too many parts to explain, you should know what it means, what type of exploit are and you should have a basic understanding of C++ and C# (just for this exploit) and I have a life.

1. update

If we just do static patch, when eg seb updates from version 3.10 to 3.11, the "file offset" or in other words the location of the code moves so the statically patched dll will crash the new version or not work in general. So for this reason injection is the better choice cause it's not going to be perfect and there would be a time that the injector might not work in things like if the method is changed in the next update or when there are things like antivirus it will completely get quarantined or removed from 'your' computer cause it's counted as a malicious software aka exploit since it

the url (the website you're visiting upon seb opening) in config and sends it to a server and the server validates from there. why just url, you asked? cause all except url are already used to make the bek you see in the configuration, so url is only used in runtime just before it gets sent to the server.

ALSO NOTE: you can still work on it by going the hard way... what I mean by that is you can get the hash the moment it gets set up in the configuration using byte array and then immediately modify the modded dll that you made and make it send the hardcoded hash by calculating the final hash and then replacing the original dll with the modded one but this is as it sounds and it's soooo so hard that I even thought I was dumb for thinking that would work in the first place (it can btw) but anyway as long as you want to learn here is the hash generator from the url and BEK hash that I made for some useless reason but I'm sure one of you can get something from it:

```
import hashlib

# URL is the website you're visiting or opening in SEB
url = "https://example.com"

# raw_bek is the one you see on the configuration
raw_bek = "b84a05d35ed35268b0cb28b4ba120db6ec773f26250aed64e3953821b9e32903"

combined_strings = url + raw_bek
final_hash = hashlib.sha256(combined_strings.encode('utf-8')).hexdigest()

print(final_hash)
```

so now let's go to injection and the interesting and obvious thing about this is that the file on the disk remains the same, we only change the code in the computer's ram after the program has already started and passed its initial integrity checks but in the future update this can get prevented if they (seb's dev) added a runtime integrity checking which is still bypassable BUT still it's going to make it harder, but I don't think it's going to happen any time soon and may be not at all.

Okay, now that you have some understanding of why we're doing this, let's go to section 3.1 where I explains what the injector does and how it does it.

3.1 The Injector

The injector is a part of this exploit which loads a payload (in our case a native dll) into the target process. It operates by finding a running 'version' of SEB and forcing it to load our custom native library. To achieve this, the injector does three things or let just say phases cause I feel like that might express it better... anyway those are finding the process, preparing the memory, and APC (asynchronous procedure call) hijacking.

The injection process begins with target discovery and this discovery is implemented using the windows Toolhelp32 api where the injector takes a systemic "snapshot" of all running processes and threads using CreateToolhelp32Snapshot. It then iterates through the results, comparing the filename of every running process with 'SafeExamBrowser.exe' and once a matching pid is found, the discovery logic must go one step deeper by enumerating all threads owned by that pid to find the exact thread id and this thread will serve as the "host" for our asynchronous procedure call (APC).

Once both the process identifier and thread identifier (TID) are found, the injector move on to create an interaction with the target by opening a handle to the process using 'OpenProcess' with VERY MINIMAL required perms like 'PROCESS_VM_OPERATION' and 'PROCESS_VM_WRITE' which make sure that the injector can allocate and also modify memory inside the target without requesting excessive privileges that might and will get blocked by a windows defender or any basic AVs. it then opens a handle to the selected thread using 'OpenThread' with 'THREAD_SET_CONTEXT' access which is necessary for scheduling execution within that thread. After that the injector allocates a memory inside the seb's process using 'VirtualAllocex' and writes the path of the native loader's dll into that memory using 'WriteProcessMemory' and preping the argument that will be passed to the loader function. At this point, the injector dynamically resolves the address of 'LoadLibraryW' from kernel32.dll using 'GetModuleHandleW' and 'GetProcAddress' and if this is a bit complicated to you, I advice you to look at the code and the resources that I put... anyway, it's used for loading things like external modules into a process's address.

This injector uses 'QueueUserAPC' instead of the most obvious and detectable function 'CreateRemoteThread' to execute 'LoadLibraryW' with the injected dll path. This method can evade common AVs and windows defender cause the execution occurs within an existing thread rather than creating a new one (which is what most AV expected from an exploit).

After queuing the APC, the injector closes all opened handles to clean up things but not the allocated memory since seb's thread may not have executed the call yet. It then waits for the target process to exit and at the same time the injector runs in a continuous loop, monitoring for the target application to restart and automatically reinjecting when it does so it can be persists for obvious reasons.

NOTE: I'm not and I will not explain every line of code, even though I said I will consider newcomers reading this and do not expect a full on explanation from me BUT I'll give you the documentation you need for each function that I think most don't know and honestly, even if I did explain everything, I will not be going to give you anything better than the resources/doc/spec itself.

3.2 Payload Mapping and 3.3 The Patch will not be explained here, read the comments of the code and use the resources that I put.

Thanks for giving it time to read ;)

